

Federated Learning on Heterogeneous Sensor Data

By David, Jonathan Truong Son

Federated Learning Implementation

Libraries/Methods

- Pytorch, CNN, 15 clients, Stochastic Gradient Descent (SGD)

Datasets

- MNIST: handwritten digits(0-9), 60k, 28x28 pixels, grayscale

Levels of Heterogeneity

- Determined by the bias variable (0-1)
- Affects the data distribution to clients

```
# assign a data point to a group  
upper_bound = (y) * (1 - bias_weight) / 9. + bias_weight  
lower_bound = (y) * (1 - bias_weight) / 9.  
rd = np.random.random_sample()
```

Level of Heterogeneity (Bias)

Fang Distribution Characteristics:

- Skews data distribution among clients based on the `bias_weight` parameter.

Logic:

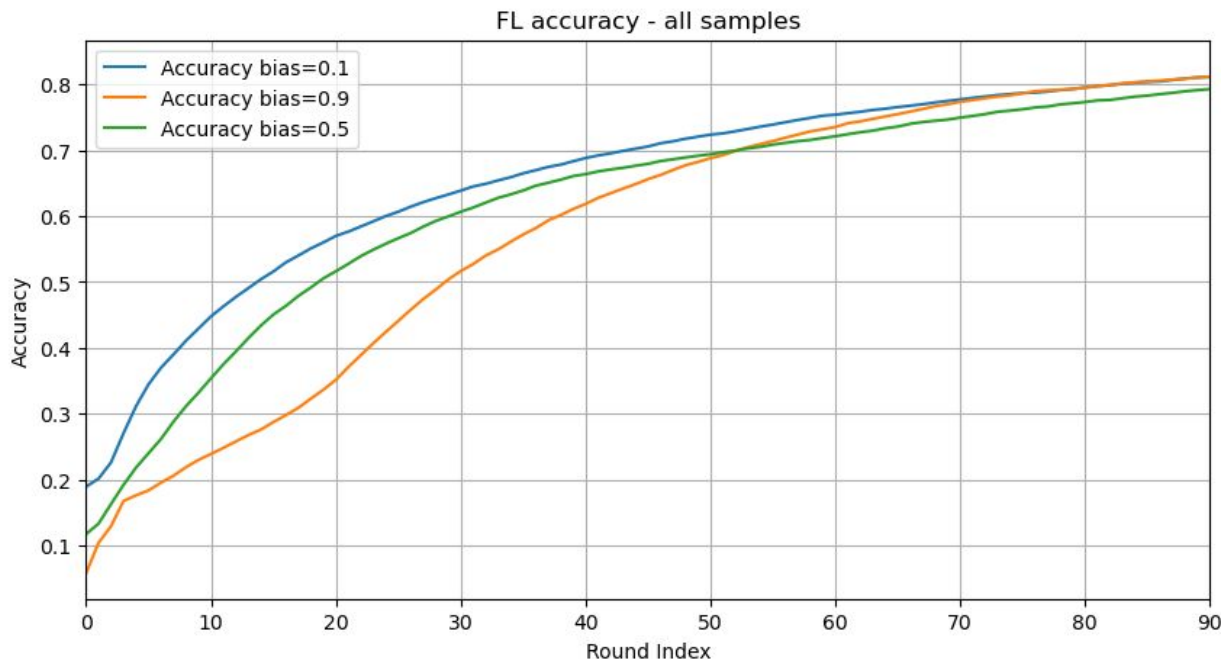
- If $rd > upper_bound$: Assign the data point to a group after y .
- If $rd < lower_bound$: Assign the data point to a group before y .
- Otherwise: Assign the data point directly to group y .

$Bias_weight = k$ ($0 < k < 1$):

- $(k*100)\%$ of the data client decision is influenced by the label y .
 - $1 - (k*100)\%$ is determined by randomness.
- 

Accuracy at different Data distribution

Bias	Group tendency	Data Distribution Type
0.1	Mostly random	Close to IID
0.5	Balanced between fixed label and randomness	half-IID
0.9	Strong label biased	Non-IID

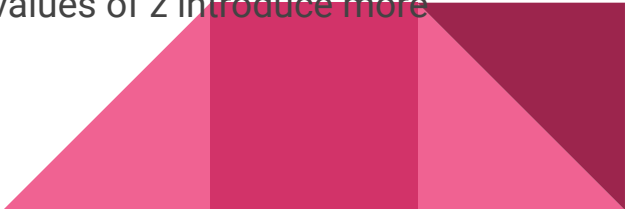


Attacks

- Full knowledge Static AGR Attack

- The attacker has full knowledge of the gradient from all client
- Depend on the number of compromised clients (number of client be attacked)
- the adversary manipulates the model updates in such a way that it can effectively change the global model after aggregation, without being easily detected

- LIE Attack

- The LIE attack modifies the gradient aggregation by using the average and standard deviation of the collected gradients.
 - The attack's strength is controlled by the parameter z . Larger values of z introduce more perturbation into the gradient.
- 

Attack codes

```
def full_trim(v, f):
    vi_shape = v[0].unsqueeze(0).T.shape
    v_tran = v.T

    maximum_dim = torch.max(v_tran, dim=1)
    maximum_dim = maximum_dim[0].reshape(vi_shape)
    minimum_dim = torch.min(v_tran, dim=1)
    minimum_dim = minimum_dim[0].reshape(vi_shape)
    direction = torch.sign(torch.sum(v_tran, dim=-1, keepdims=True))
    directed_dim = (direction > 0) * minimum_dim + (direction < 0) * maximum_dim

    for i in range(f):
        # apply attack to compromised worker devices with randomness
        random_12 = 2
        tmp = directed_dim * ((direction * directed_dim > 0) / random_12 + (direction * directed_dim < 0) * random_12)
        tmp = tmp.squeeze()
        v[i] = tmp
    return v
```

#LIE Attack

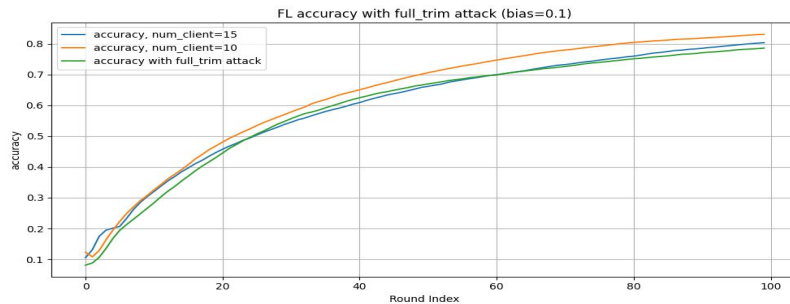
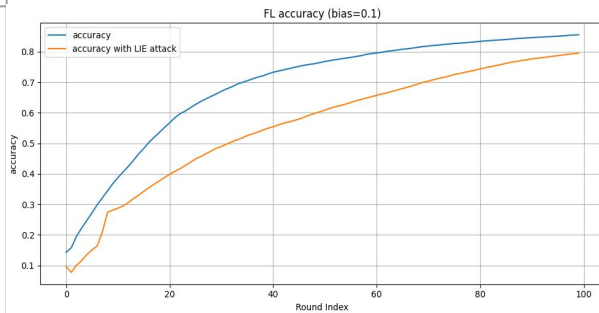
```
def lie_attack(all_updates, z):
    avg = torch.mean(all_updates, dim=0)
    std = torch.std(all_updates, dim=0)
    return avg + z * std
```

Bias

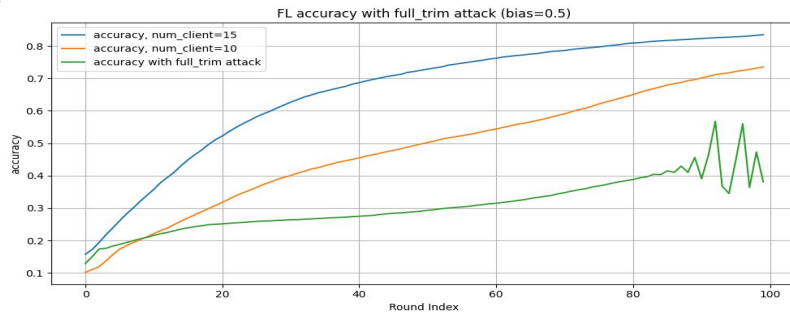
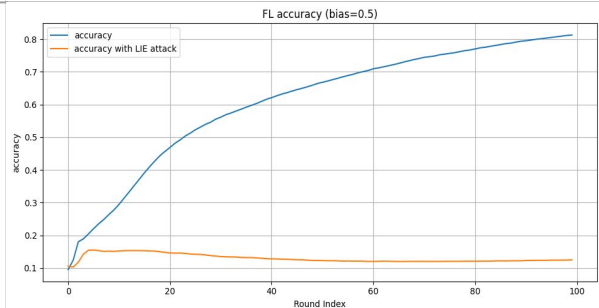
Comparison (all, LIE attack)

Comparison (all, 10/15, FKTrim attack)

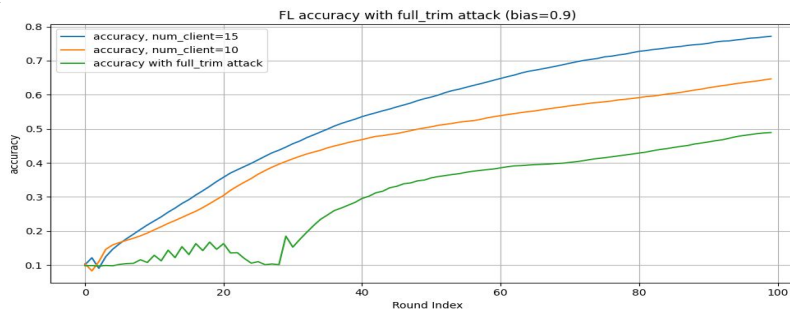
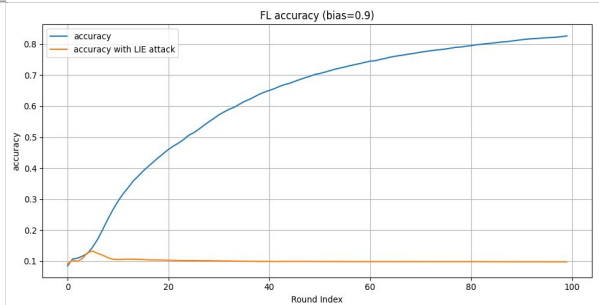
0.1



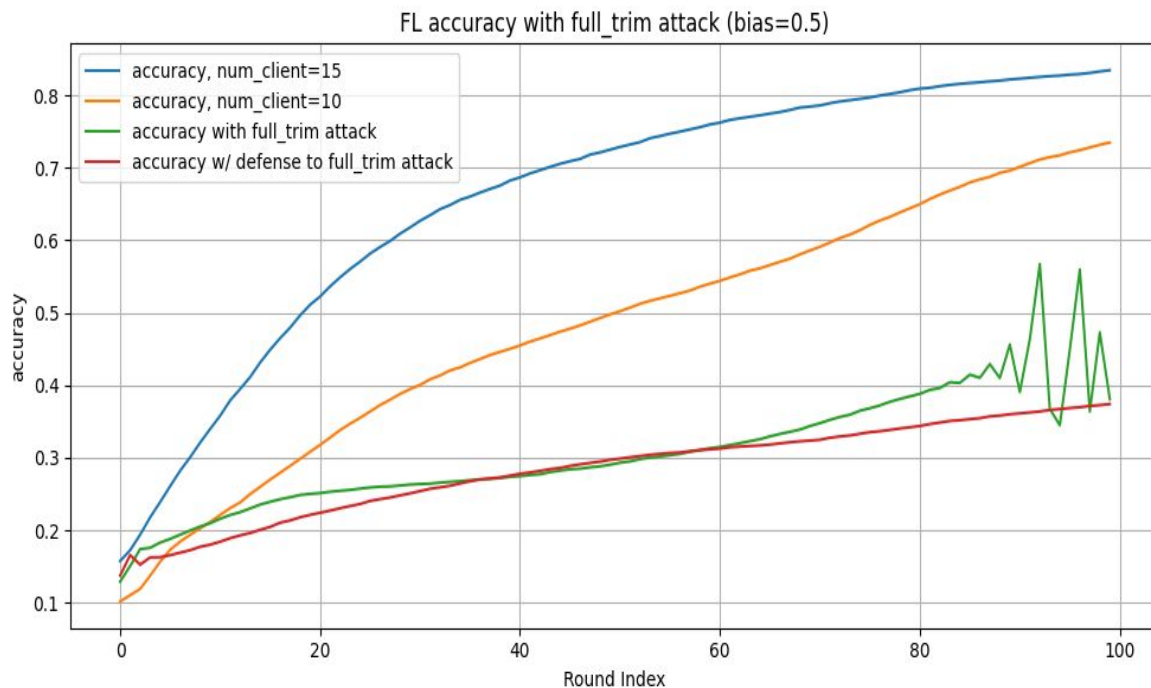
0.5



0.9



tr_mean for prevent full_trim attack (bias = 0.5)

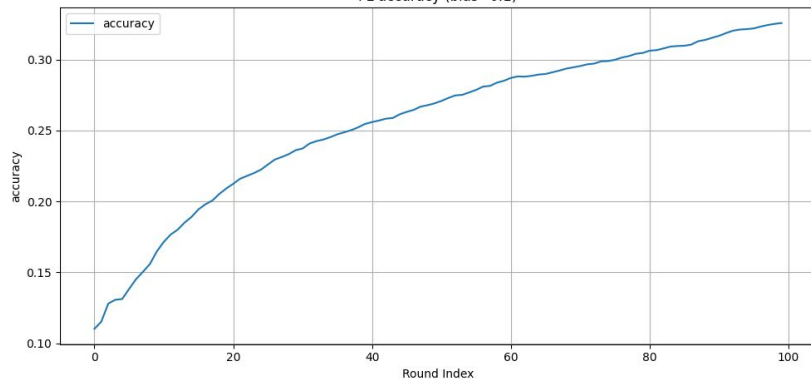


Thank You.

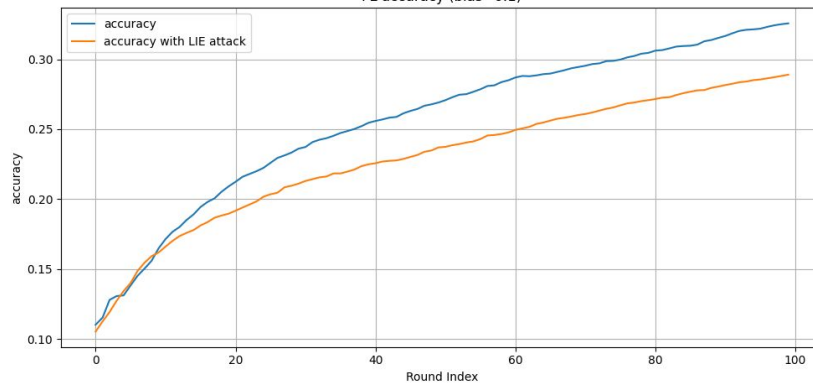


CIFAR10, bias=0.1

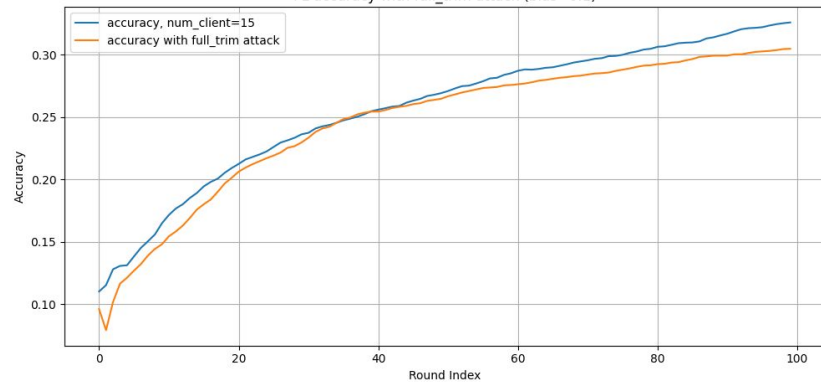
FL accuracy (bias=0.1)



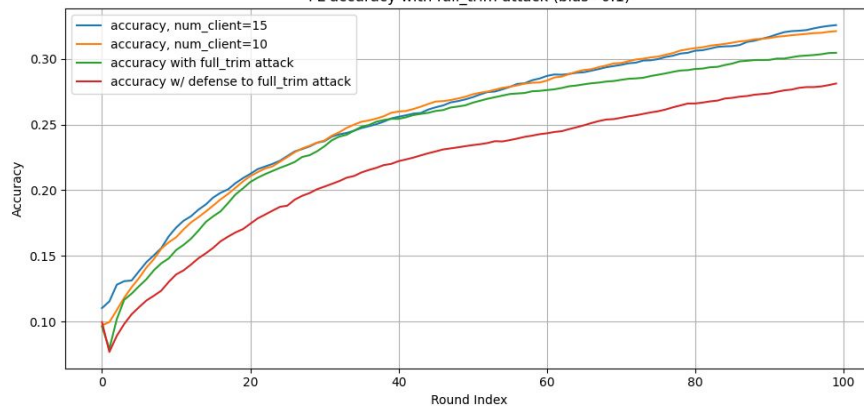
FL accuracy (bias=0.1)



FL accuracy with full_trim attack (bias=0.1)

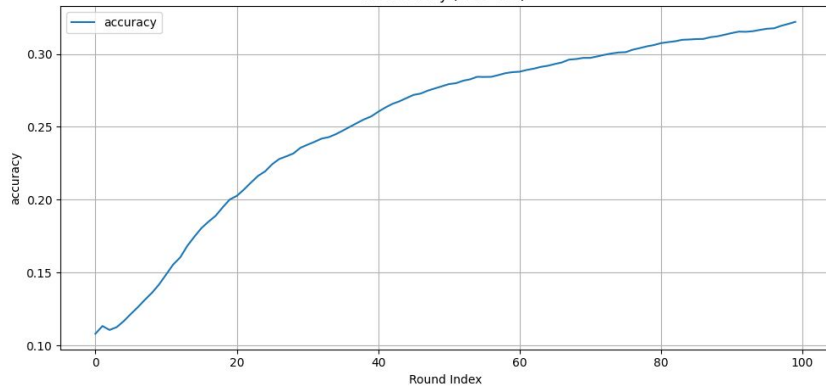


FL accuracy with full_trim attack (bias=0.1)

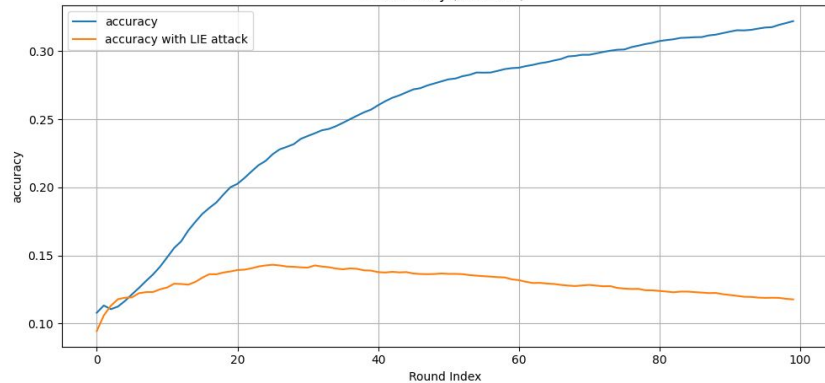


CIFAR10, bias=0.5

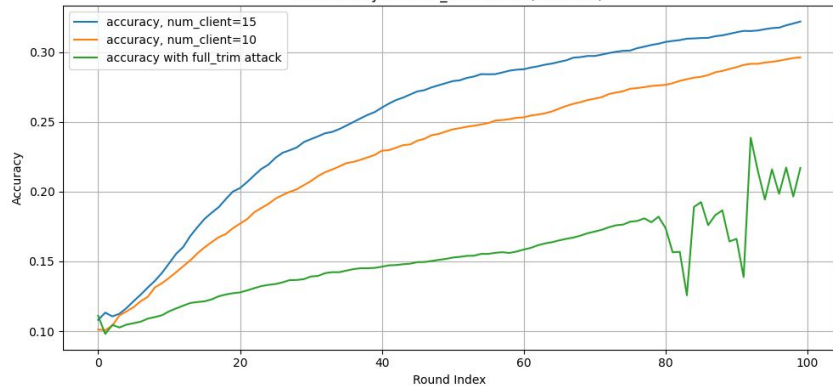
FL accuracy (bias=0.5)



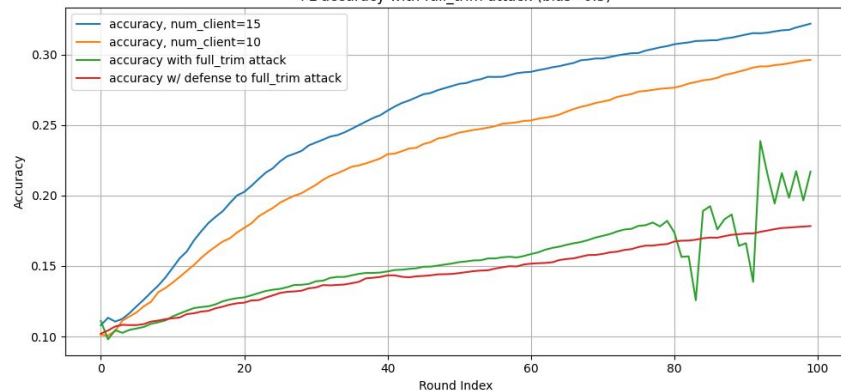
FL accuracy (bias=0.5)



FL accuracy with full_trim attack (bias=0.1)

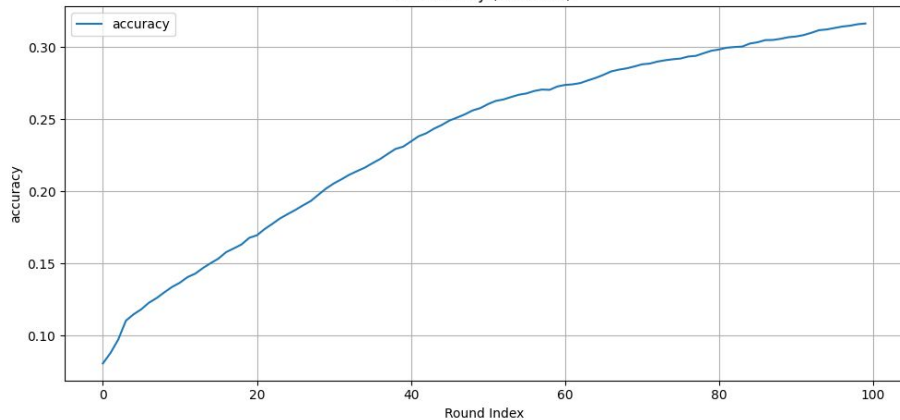


FL accuracy with full_trim attack (bias=0.5)

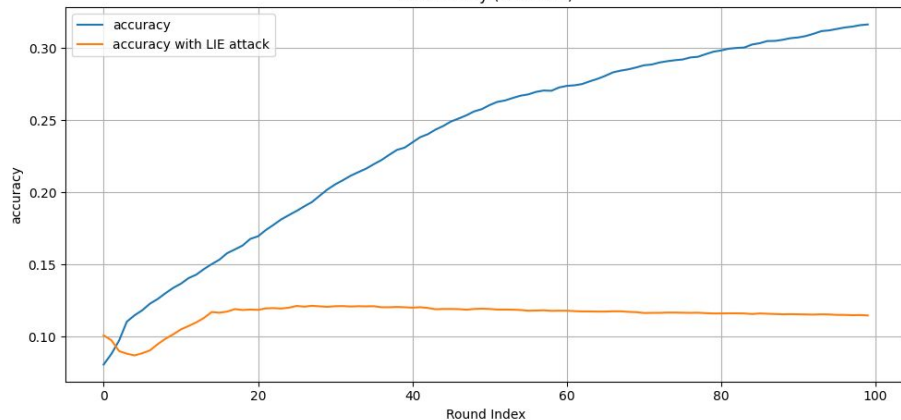


CIFAR10, bias=0.9

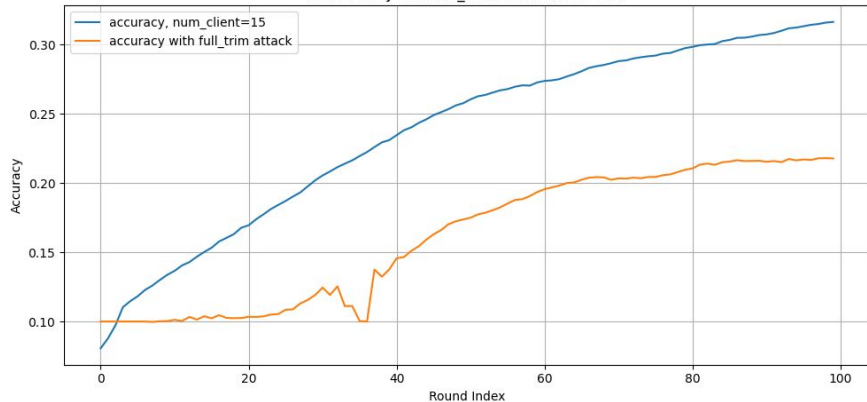
FL accuracy (bias=0.9)



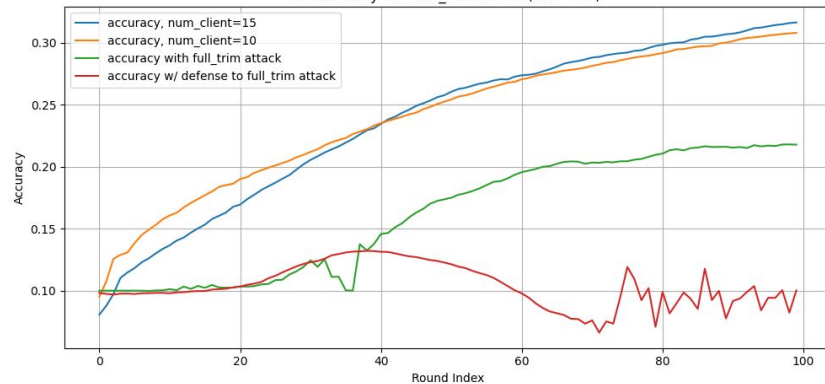
FL accuracy (bias=0.9)



FL accuracy with full_trim attack (bias=0.9)



FL accuracy with full_trim attack (bias=0.9)

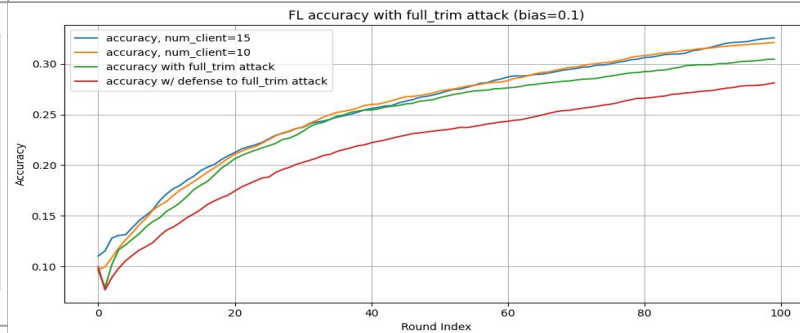
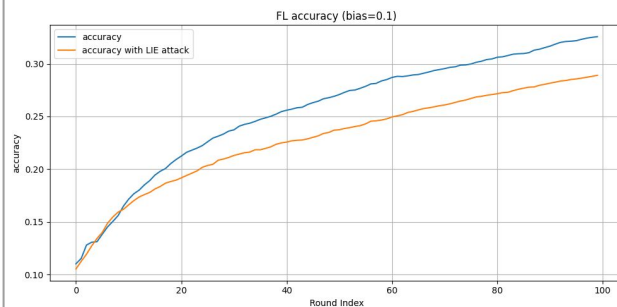


Bias

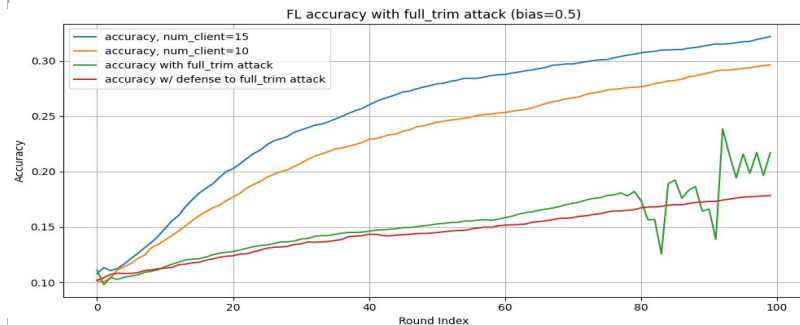
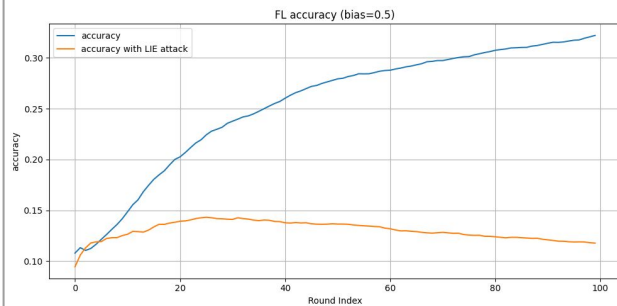
Comparison (all, LIE attack)

Comparison (all, 10/15, FKTrim attack)

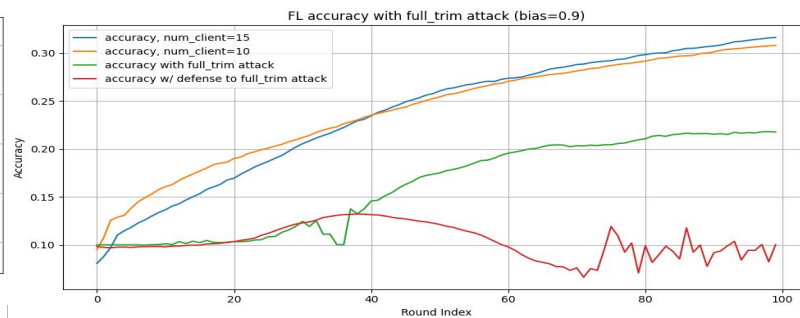
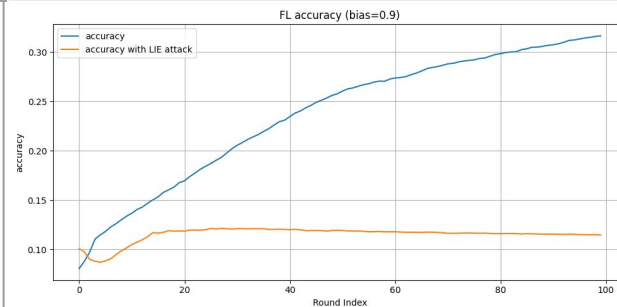
0.1



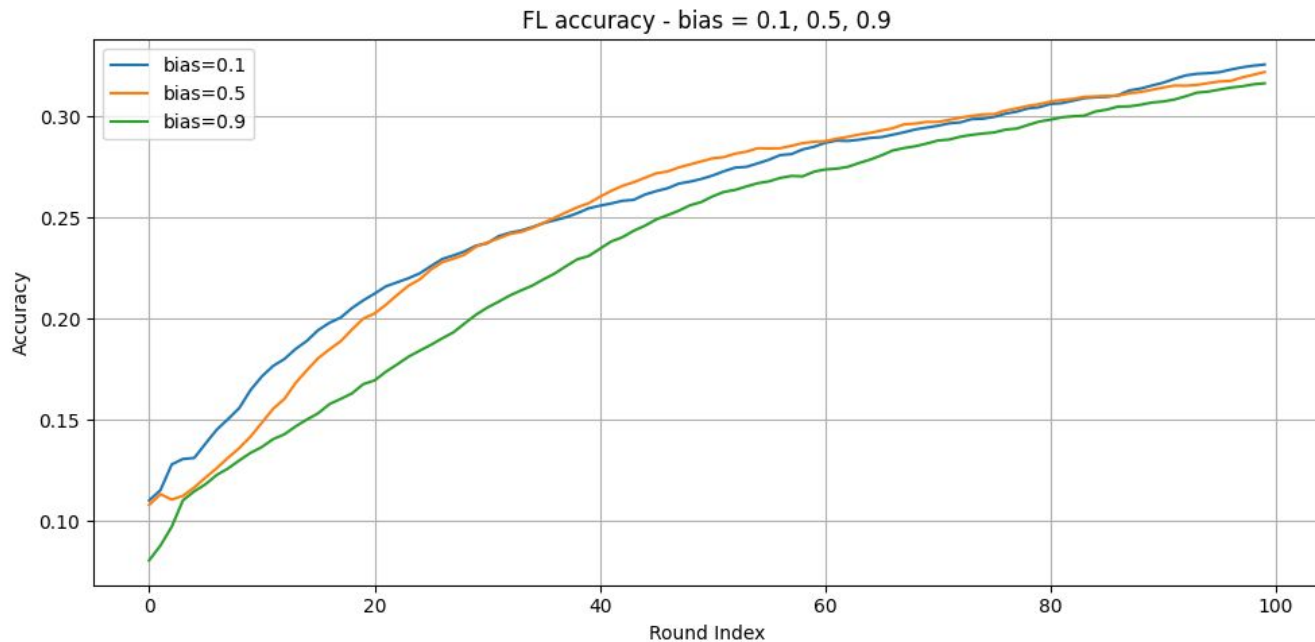
0.5



0.9



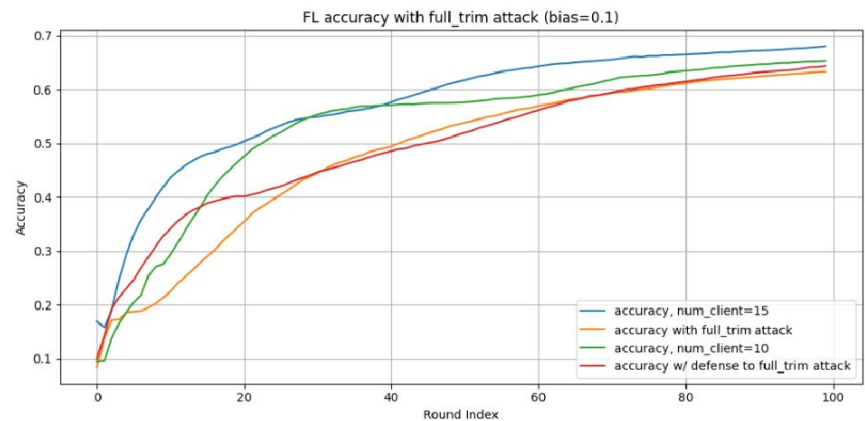
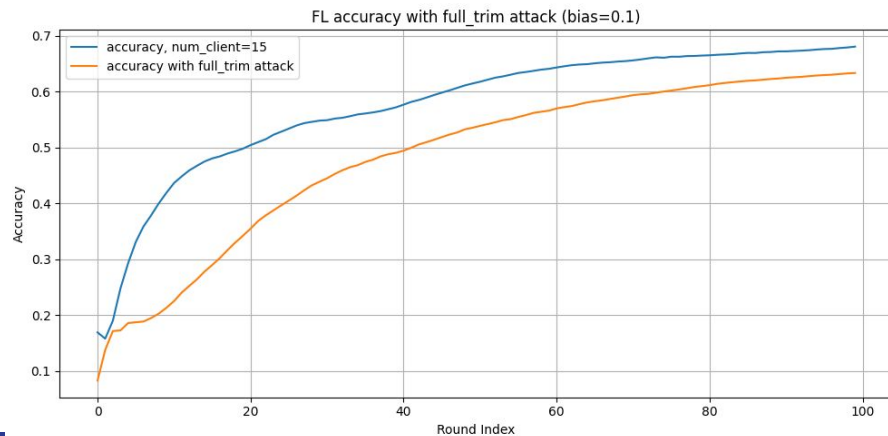
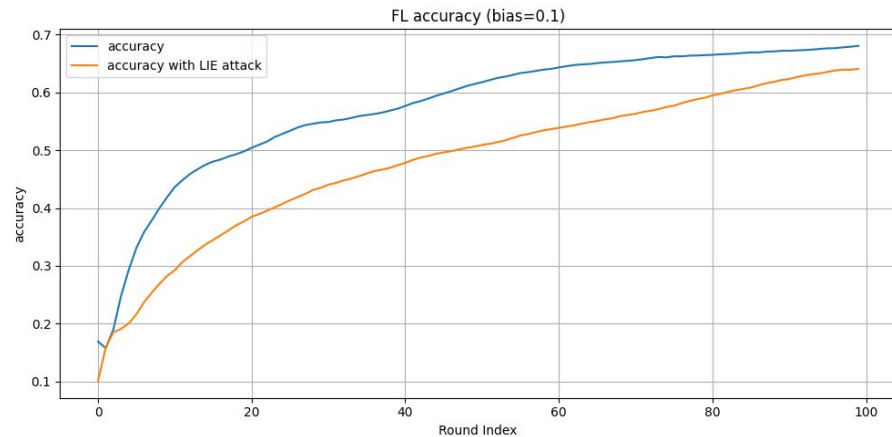
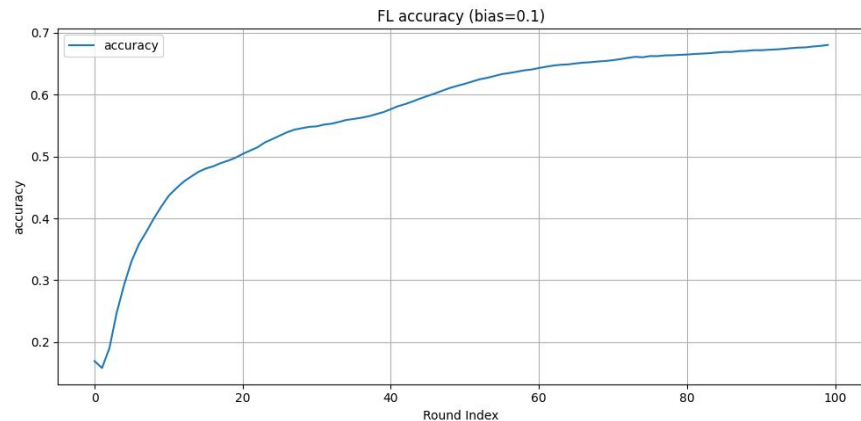
CIFAR10, bias = 0.1, 0.5, 0.9



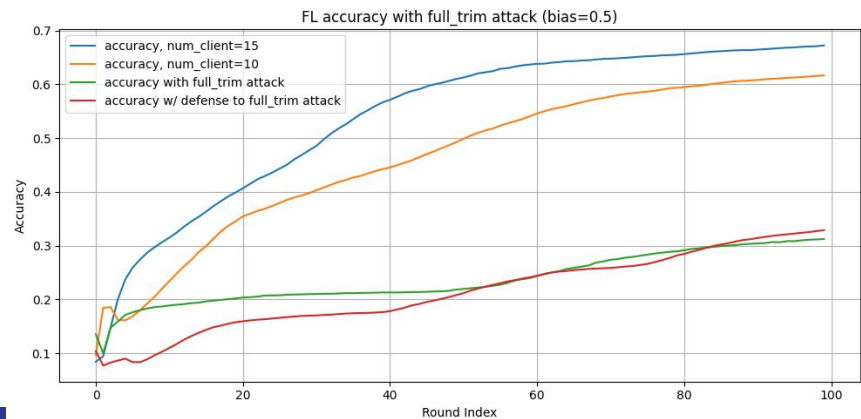
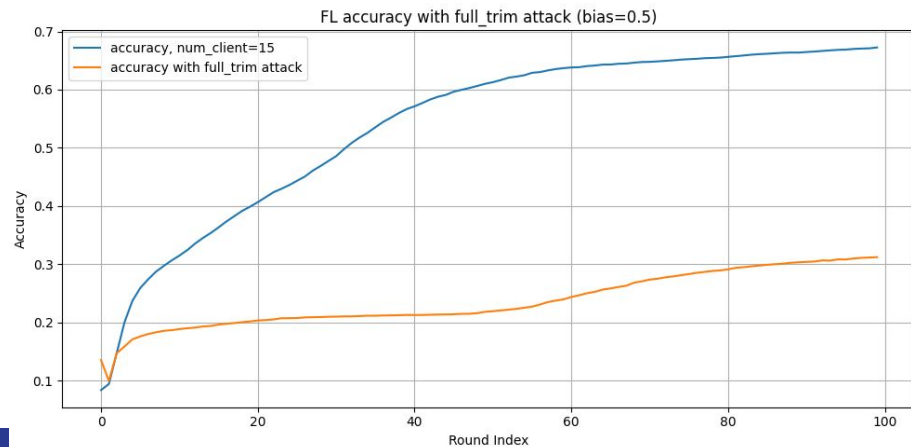
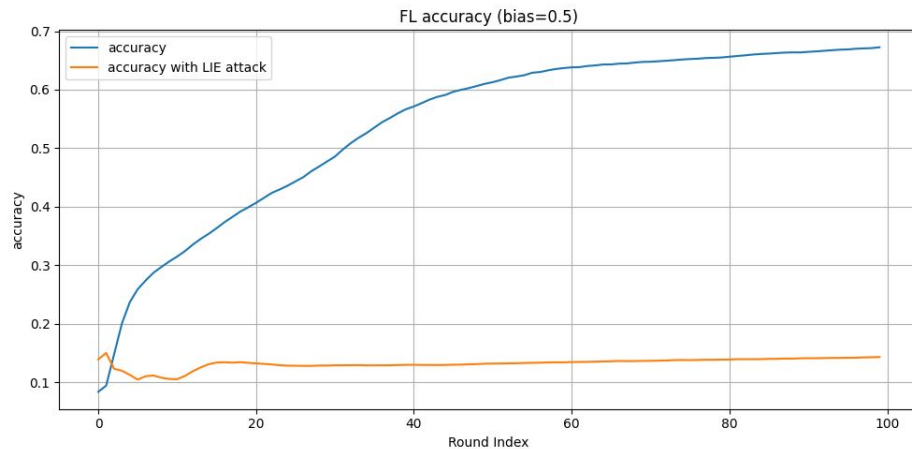
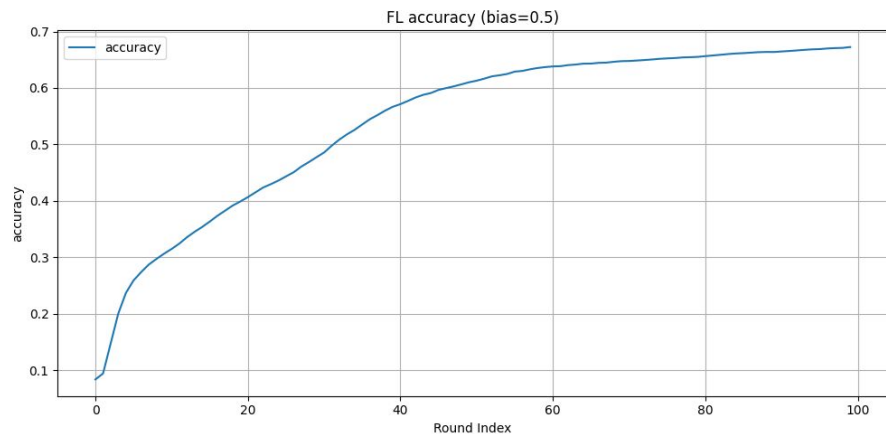




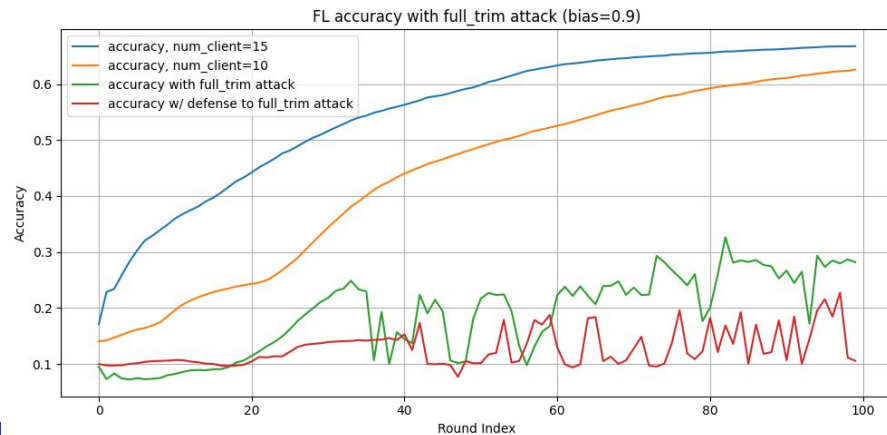
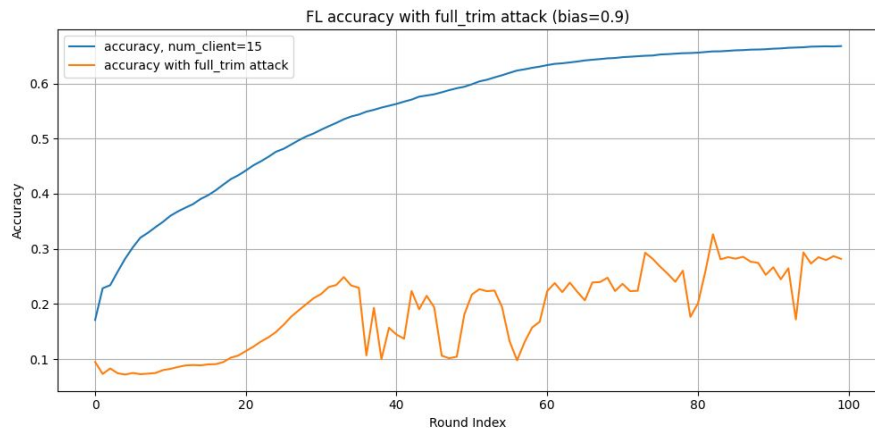
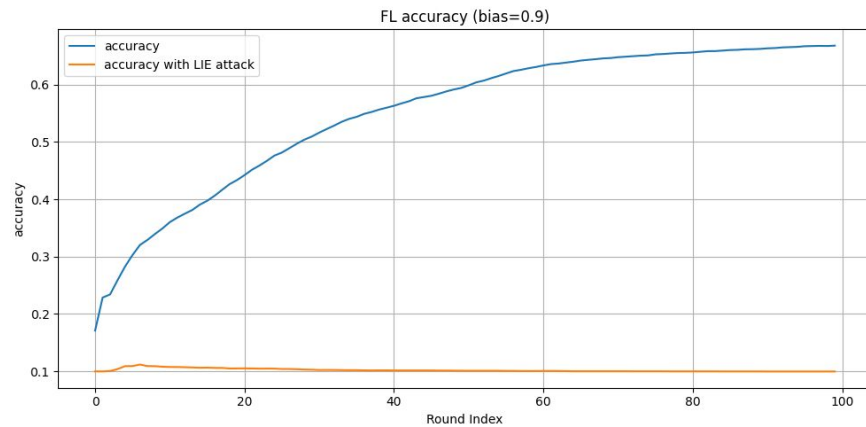
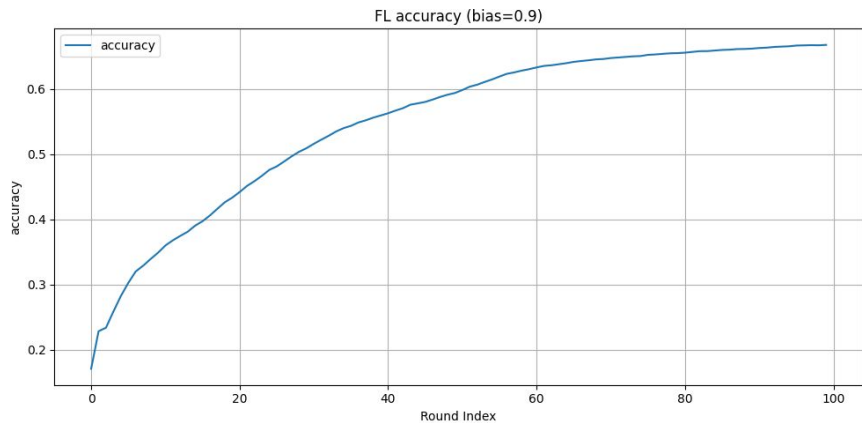
Fashion MNIST, bias = 0.1



Fashion MNIST, bias = 0.5



Fashion MNIST, bias = 0.9

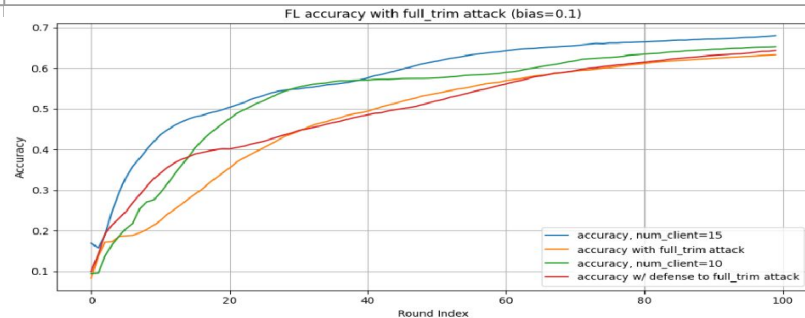
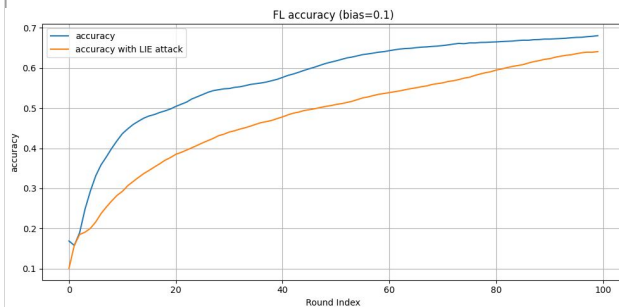


Bias

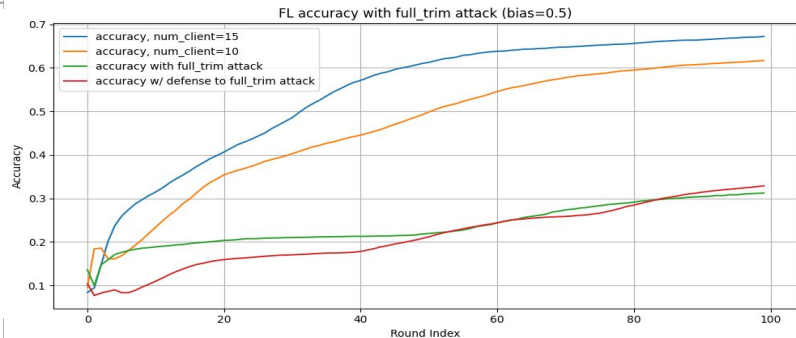
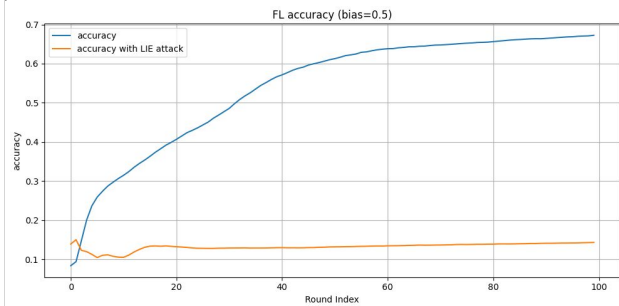
Comparison (all, LIE attack)

Comparison (all, 10/15, FKTrim attack)

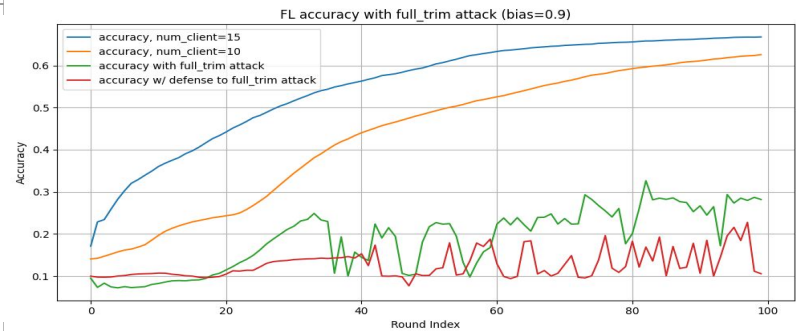
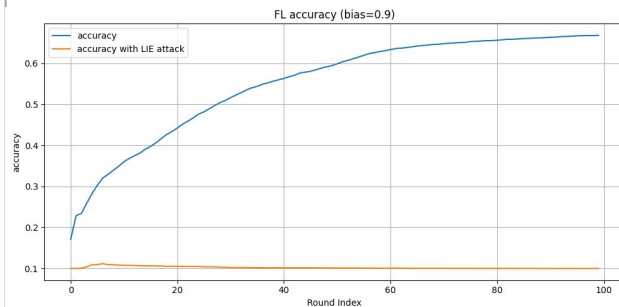
0.1



0.5



0.9



Fashion MNIST, bias = 0.1, 0.5, 0.9

